

ERP BUSINESS LOGIC FOR AGENTIC AI

ARCHITECTURE & IMPLEMENTATION GUIDE

Semantic Memory · Tool Registry · Execution Guardrails · AI Foundry

SAP CDS Views · Unity Catalog Metadata · Azure APIM · Dual-Layer Safety

Ball Corporation — Technical Architecture & Design Document

Confidential · May 2026

TABLE OF CONTENTS

1. The Problem — Why Raw Data Is Not Enough for AI Agents
2. Extract Logic & Context from SAP (Not Just Raw Tables)
3. Semantic Memory Layer in Unity Catalog
4. Actionable Tool Registry for Agent Execution
5. Dual-Layer Execution Guardrails
6. End-to-End Agentic AI Architecture
7. CDS View Extraction Architecture
8. SAP BDC + Databricks Delta Sharing Integration
9. Agent Governance Data Model
10. Azure AI Foundry Cross-Platform Integration
11. Implementation Roadmap & Checklist

1. THE PROBLEM — WHY RAW DATA IS NOT ENOUGH

Bringing raw data into a data lake is only half the battle. If AI agents do not understand the underlying ERP business logic — calculations, statuses, custom workflows, and system metadata — they will hallucinate or generate invalid queries against your enterprise data.

The Core Challenge: SAP tables use cryptic German 4-letter field names (MANDT, BUKRS, MATNR), rely on decades of custom ABAP logic for calculations, and encode business rules across hundreds of interconnected tables. An AI agent querying raw tables has no way to know that NETWR requires currency conversion via WAERK, or that a sales order status depends on a complex chain of delivery, billing, and payment conditions.

1.1 What Goes Wrong Without Business Logic

Scenario	Without Business Logic	With Business Logic
Agent queries sales revenue	Pulls raw NETWR from VBAP without currency conversion → Wrong revenue figures	Uses CDS View with pre-calculated net_value in reporting currency
Agent checks order status	Reads GBSTK field = "C" → Has no idea what "C" means	Semantic layer maps "C" = "Fully Processed" with business context
Agent creates a work order	Attempts raw INSERT into SAP tables → data corruption	Uses registered Tool via Azure APIM → safe OData call with guardrails
Agent analyzes customer credit	Cannot access custom Z_* ABAP credit tier logic	Custom rules exposed as OData entities → captured in semantic metadata

Figure 1.1 — Impact of Missing Business Logic on AI Agent Accuracy

1.2 The Four Pillars of ERP Intelligence for AI

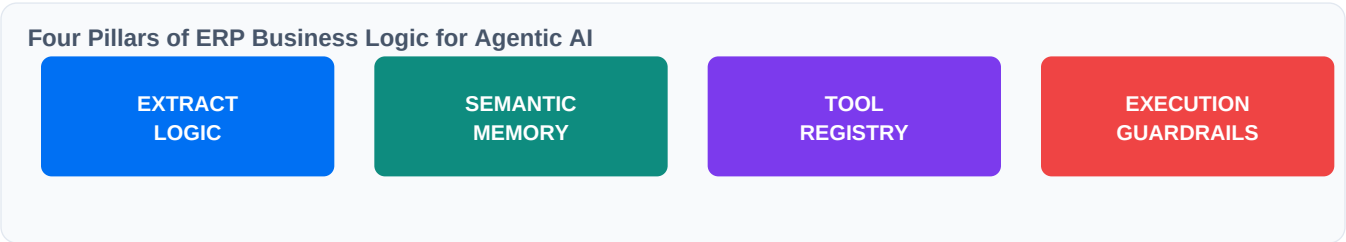


Figure 1.2 — Four Pillars Architecture

2. EXTRACT LOGIC & CONTEXT FROM SAP

Do not recreate decades of ERP logic in Databricks manually. Extract the semantic intelligence directly from the SAP application layer using native capabilities. The goal is to capture pre-built joins, authorization parameters, calculated fields, and business rules as they exist in SAP.

2.1 Strategy A — CDS Views via ODP Framework

Avoid pulling raw tables like VBAK or VBAP whenever possible. Pull from SAP Core Data Services (CDS) Views via the Operational Data Provisioning (ODP) framework. CDS views inherently contain pre-built joins, authorization parameters, and calculated fields (net values, currency conversions, material statuses) directly inside the payload.

SAP Module	Raw Tables (Avoid)	CDS View (Recommended)	Built-In Logic
Finance (FI)	BKPF, BSEG	I_JournalEntry I_GLAccountLineItem	Currency conversion, fiscal year mapping
Sales (SD)	VBAK, VBAP	I_SalesDocument I_SalesDocumentItem	Net value calc, status resolution, partner data
Materials (MM)	EKKO, EKPO	I_PurchasingDocument I_PurchaseOrderItem	Price conditions, quantity conversions
Production (PP)	AUFK, AFKO	I_ProductionOrder I_MfgOrderOperation	Operation scheduling, scrap calculation
Quality (QM)	QALS, QAPO	I_InspectionLot	Inspection results, usage decisions

Figure 2.1 — CDS View Extraction Matrix by SAP Module

2.2 Strategy B — SAP Business Data Cloud (BDC) Delta Sharing

The native integration between SAP Business Data Cloud (BDC) and Databricks Unity Catalog automatically syncs semantic metadata at the table level. When you mount an SAP BDC share, Unity Catalog natively displays business-friendly field labels, table descriptions, and context — the AI agent instantly sees 'Sales Order Number' instead of VBELN.

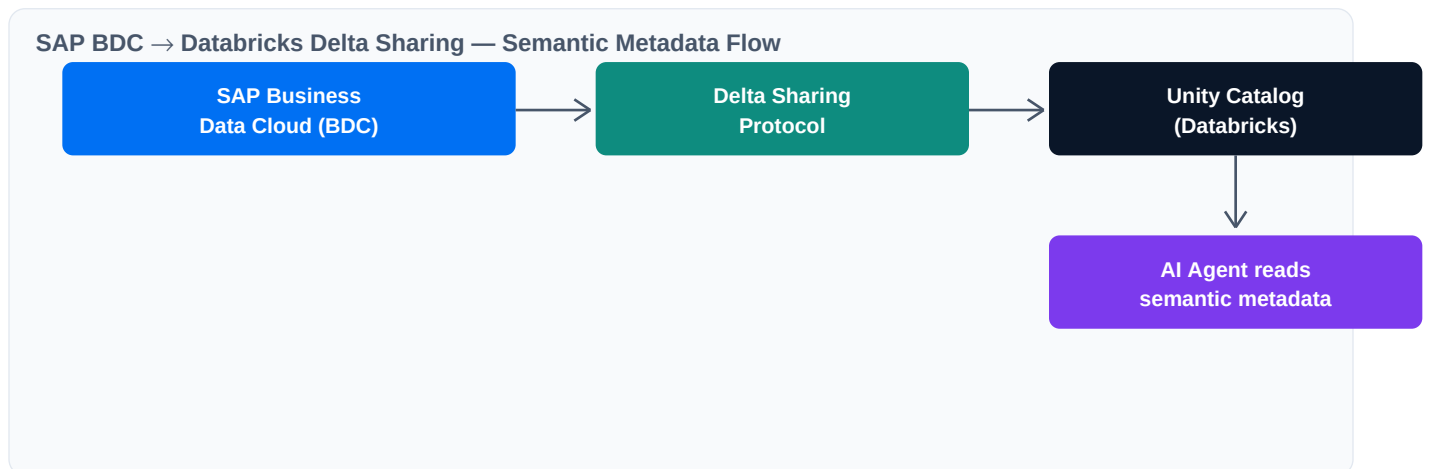


Figure 2.2 — BDC Delta Sharing Architecture

2.3 Strategy C — Custom ABAP Rules as OData Entities

If your legacy system relies on custom ABAP function modules (Z_* code or user exits) to calculate business values like dynamic customer credit tiers, expose those calculations via custom OData entities so they can be captured by your ingestion pipeline.

- **Step 1:** Identify all Z_* function modules that calculate business-critical values
- **Step 2:** Wrap each function module in a custom CDS View or OData V4 entity
- **Step 3:** Register the OData endpoint in ADF for scheduled extraction to Bronze layer
- **Step 4:** Document the business rule in Unity Catalog column comments and YAML manifests

3. SEMANTIC MEMORY LAYER IN UNITY CATALOG

An Agentic AI agent queries catalog metadata before writing a SQL query or invoking a tool. The Semantic Memory Layer enriches Unity Catalog with business context so the AI agent understands not just table structures, but business meaning, relationships, and rules.

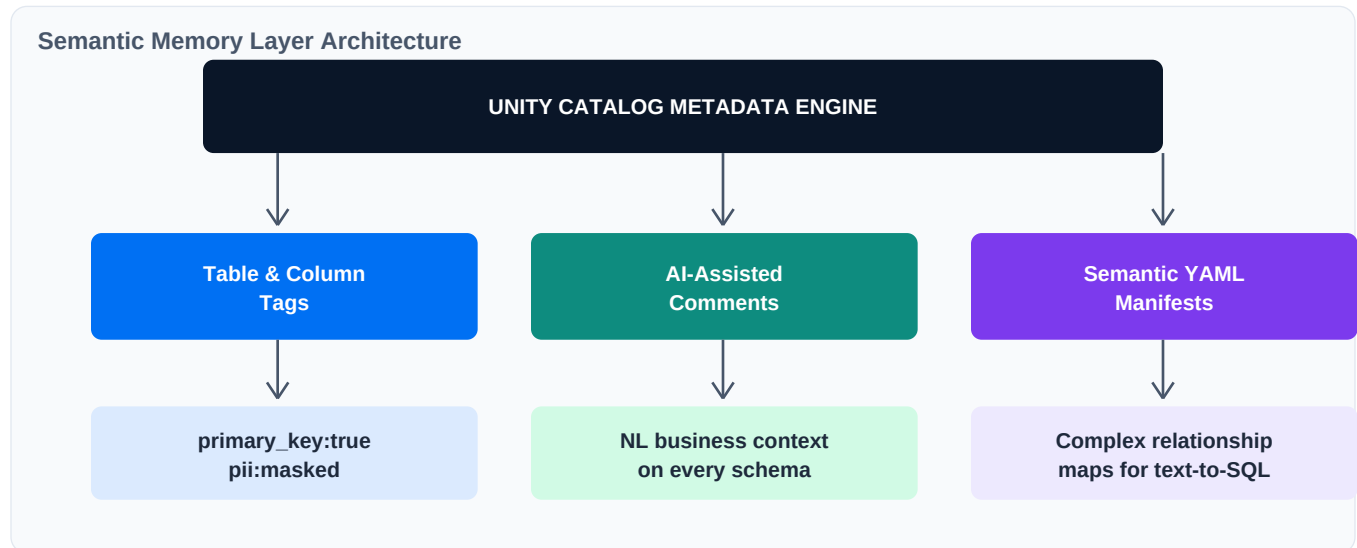


Figure 3.1 — Unity Catalog Semantic Memory Architecture

3.1 AI-Assisted Metadata Enrichment

Use Unity Catalog's native generative AI documentation feature to automatically build markdown summaries, define column functions, and map dependencies based on execution logs and historical usage patterns.

- **What It Does:** Auto-generates column descriptions from query patterns and data profiling
- **Lineage Mapping:** Maps table dependencies and identifies upstream/downstream relationships
- **Glossary Generation:** Creates business glossary entries that AI agents can reference at query time

3.2 Semantic Schema Guides (YAML Data Contracts)

Large Language Models reason best when provided with structured guides. Store a master configuration table in your governance catalog containing YAML blocks that outline table relationships for the agent:

```
table_name: sap_domain_sales.sales_products.global_sd_orders
description: "Contains all customer sales order items and headers across NA and EU."
primary_key: "sales_document_id"
joins:
  - target_table: "sap_domain_sales.sales_products.global_sd_customers"
    on_condition: "global_sd_orders.customer_id = global_sd_customers.global_customer_id"
    description: "Join to filter sales orders by customer attributes or country."
```

Figure 3.2 — Example YAML Schema Manifest for AI Agent Consumption

3.3 Metadata Storage Design

Table / Location	Content	Consumer
------------------	---------	----------

sap_enterprise.agent_governance .schema_manifests	YAML blocks with table relationships, join conditions, business rules	AI Foundry Text-to-SQL tool at query time
sap_enterprise.agent_governance .business_glossary	Field-level descriptions, SAP-to-English mapping, domain context	RAG pipeline for context injection
sap_enterprise.agent_governance .action_api_spec	OpenAPI specifications for all registered SAP action endpoints	AI Foundry function calling framework
Unity Catalog native table/column comments	Inline markdown documentation on every schema, table, and column	Databricks Catalog Explorer + AI agents

Figure 3.3 — Agent Governance Metadata Tables

4. ACTIONABLE TOOL REGISTRY FOR AGENT EXECUTION

For an AI agent to safely execute actions (modifying data, changing statuses in SAP), it requires tool descriptions that tell Microsoft AI Foundry exactly how to call an API. Register available actions directly inside Unity Catalog as a governed, versioned registry.

4.1 Tool Registry Schema

```
CREATE TABLE sap_enterprise.agent_governance.action_tool_registry (  
  tool_name STRING,          -- Name the LLM references  
  tool_description STRING,    -- Explains WHEN the agent should use this tool  
  api_endpoint STRING,        -- Target Azure APIM URL linked to SAP Gateway  
  http_method STRING,         -- POST, PUT, PATCH  
  required_parameters STRING, -- JSON Schema describing exact data formats  
  guardrail_policy_id STRING -- Link to safety compliance tables  
);
```

Figure 4.1 — Tool Registry DDL

4.2 Writing Effective Tool Descriptions

When Microsoft AI Foundry parses the tool registry, the `tool_description` field serves as the operational prompt for the model. The quality of this description directly determines whether the agent invokes the right tool at the right time.

Bad Description: "Updates SAP tables."

Good Agent Description: "Use this tool when a user explicitly requests to clear a financial credit block on an order. Requires input variables: `sales_order_id`, `reason_code`, and `user_id`. Do not invoke if the transaction value exceeds \$50,000 without prompting for supervisor authorization first."

4.3 Tool Registry Examples

Tool Name	Description (for LLM)	API Endpoint	Method
clear_credit_block	Clear financial credit hold on sales order. Requires <code>sales_order_id</code> , <code>reason_code</code> , <code>user_id</code> . Block if > \$50K without supervisor approval.	/sap/opu/odata/ clear_credit	PATCH
create_maintenance_order	Create PM work order for equipment breakdown. Requires <code>equipment_id</code> , <code>priority</code> , <code>description</code> . Auto-assigns planner group from master data.	/sap/opu/odata/ create_pm_order	POST
update_delivery_date	Reschedule delivery date on open sales order. Requires <code>order_id</code> , <code>new_date</code> . Cannot reschedule if goods issue already posted.	/sap/opu/odata/ update_delivery	PUT
release_purchase_order	Approve and release blocked PO. Requires <code>po_number</code> , <code>approver_id</code> . Block if total > budget threshold defined in <code>agent_execution_limits</code> .	/sap/opu/odata/ release_po	PATCH

Figure 4.2 — Sample Tool Registry Entries

5. DUAL-LAYER EXECUTION GUARDRAILS

When bridging the gap between an AI agent and your ERP system, you must prevent prompt injections from changing financial data without authorization. The dual-layer pattern implements safety checks at both the Databricks and Azure layers.

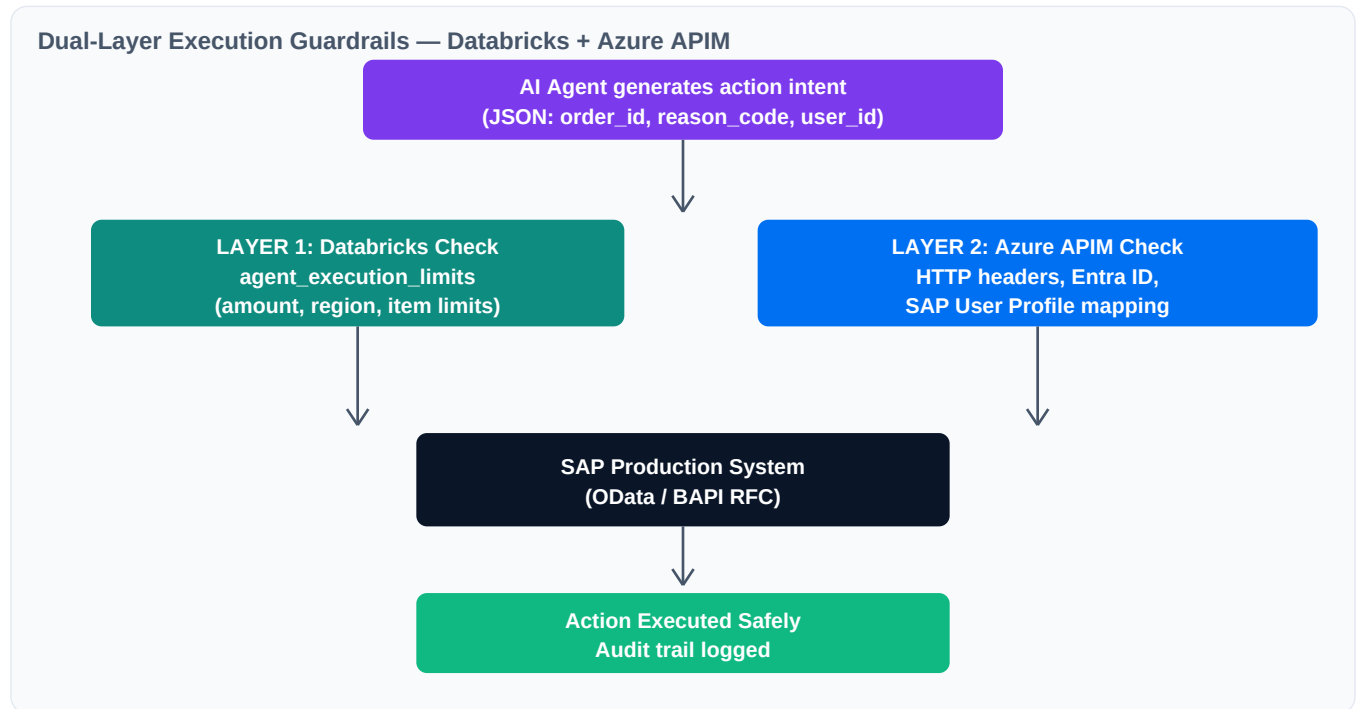


Figure 5.1 — Dual-Layer Guardrails Architecture

5.1 Layer 1 — Databricks Metadata Check

When an agent triggers an action, it must execute a native Databricks Python validation function first. This function queries the agent_execution_limits matrix inside Unity Catalog to verify that the parameters align with corporate data policies.

Policy Parameter	Threshold	Action If Exceeded
Transaction Amount	< \$50,000	Require supervisor_approval flag
Region Restrictions	NA, EU only	Block APAC transactions entirely
Daily Transaction Limit	< 100 per agent session	Pause agent, alert security team
Sensitive Fields	PII columns masked	Reject any query exposing raw PII
Time Window	Business hours only	Queue after-hours actions for review

Figure 5.2 — Agent Execution Limits Matrix

5.2 Layer 2 — Azure APIM Gateway Check

- **Policy Injection:** Inject structural validation policies into the inbound stream
- **Header Validation:** Cross-check that required HTTP headers are populated (Authorization, X-Request-ID, Content-Type)

- **Identity Mapping:** Verify that the user's Entra ID context successfully maps to a valid SAP User Profile before executing the write
- **Traffic Control:** Apply rate limiting, IP whitelisting, and payload size constraints at the gateway level
- **Audit Logging:** Log every action attempt (success or failure) to an audit table in Unity Catalog for compliance

6. END-TO-END AGENTIC AI ARCHITECTURE

The complete architecture connects SAP business logic extraction, semantic metadata in Unity Catalog, AI Foundry reasoning, and safe action execution through Azure APIM back to SAP production systems.

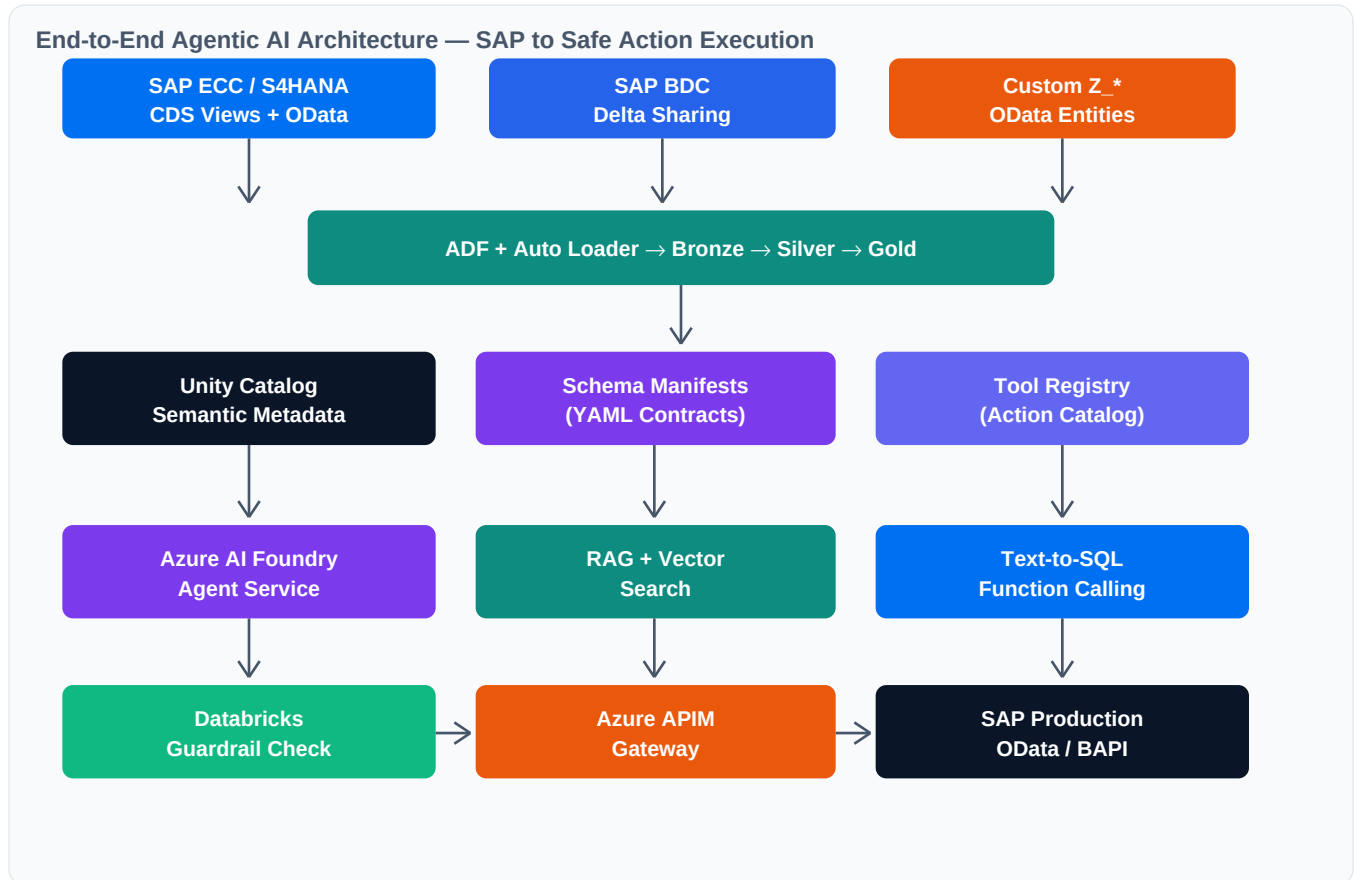


Figure 6.1 — Complete Agentic AI Architecture with ERP Business Logic

6.1 Data Flow Summary

- **Step 1 — Extract Logic:** SAP CDS Views, BDC Delta Shares, and custom OData entities extract business logic and calculated fields — not just raw tables.
- **Step 2 — Refine:** Data flows through the Medallion Lakehouse (Bronze → Silver → Gold) with business rules preserved and enriched at each layer.
- **Step 3 — Catalog:** Unity Catalog stores semantic metadata (YAML manifests, column comments, business glossary) and the Action Tool Registry.
- **Step 4 — Reason:** Azure AI Foundry agents consume metadata via RAG (for context) and Text-to-SQL (for structured queries), using registered tools for actions.
- **Step 5 — Execute Safely:** Actions pass through dual-layer guardrails (Databricks policy check + Azure APIM validation) before reaching SAP production.

7. CDS VIEW EXTRACTION ARCHITECTURE

CDS Views are the recommended extraction method because they encapsulate SAP business logic at the application layer. Unlike raw table pulls, CDS views deliver pre-joined, pre-calculated, and authorization-checked data payloads.

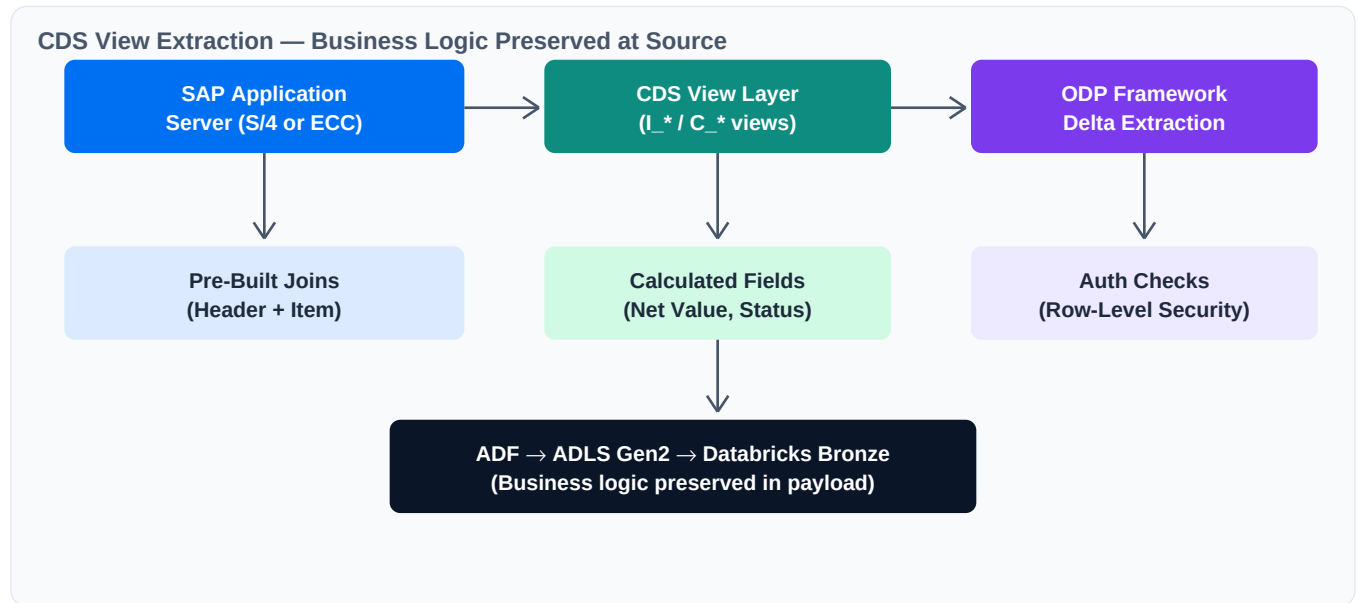


Figure 7.1 — CDS View Extraction Architecture

7.1 CDS View Types

View Type	Prefix	Purpose	Example
Basic Interface	I_	Core business object with joins and calculated fields	I_SalesDocument, I_JournalEntry
Composite	I_ (composite)	Combines multiple basic views into complex business entities	I_SalesDocumentWithPartner
Consumption	C_	Optimized for analytics and reporting with aggregations	C_SalesOrderOverview, C_ProfitCenterAnalysis
Extension	E_	Customer extensions that add custom fields to standard views	E_SalesOrder_ZCustomField

Figure 7.2 — SAP CDS View Types and Usage

8. SAP BDC + DATABRICKS DELTA SHARING INTEGRATION

SAP Business Data Cloud (BDC) provides a managed pathway for sharing SAP data products directly with Databricks Unity Catalog using the open Delta Sharing protocol. This eliminates custom ETL for metadata transfer.

8.1 Integration Architecture

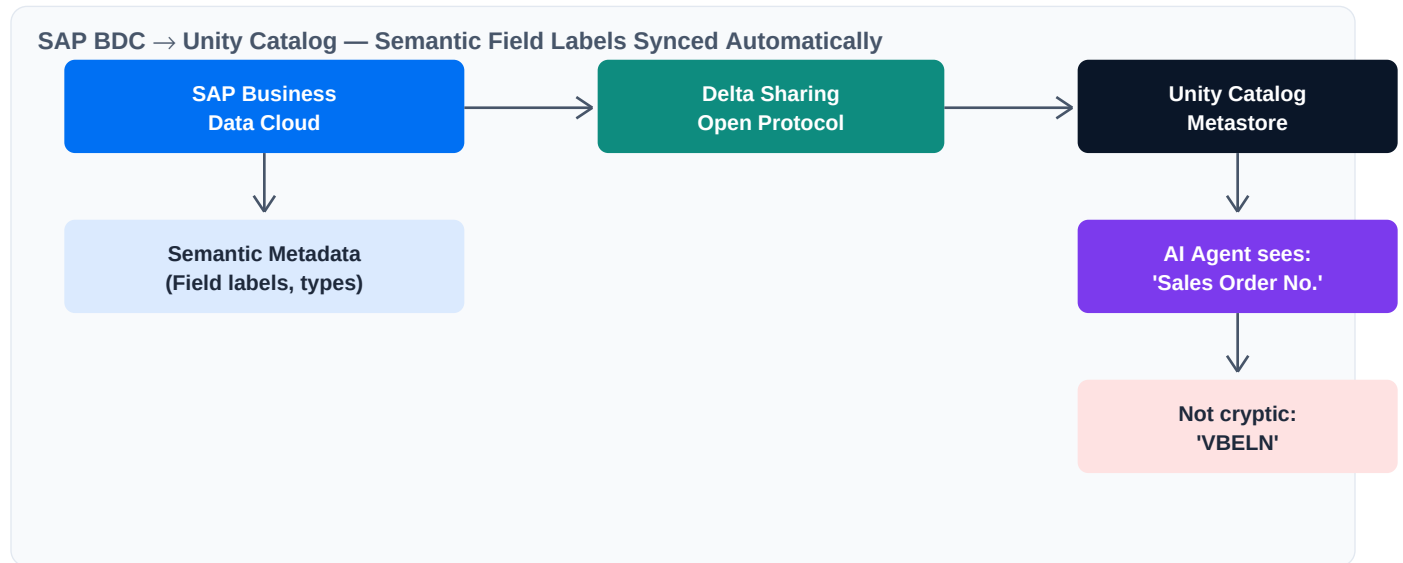


Figure 8.1 — BDC Delta Sharing with Semantic Metadata

8.2 Benefits of BDC Integration

- **Auto-Sync Metadata:** Semantic metadata (business-friendly field names, descriptions, data types) automatically syncs to Unity Catalog — no manual documentation required
- **Zero-Copy:** Data never leaves the SAP BDC storage — Databricks reads via the Delta Sharing protocol with zero-copy access
- **Schema Evolution:** Changes to SAP data models are automatically reflected in Unity Catalog, keeping AI agent context always current
- **Full Governance:** Unity Catalog governance (access grants, lineage, tags) applies to BDC-shared tables identically to native Delta tables

9. AGENT GOVERNANCE DATA MODEL

The agent governance catalog (`sap_enterprise.agent_governance`) contains all configuration, policy, and audit tables that control AI agent behavior. This is the single source of truth for what agents can see, query, and do.

Table	Purpose	Key Columns
schema_manifests	YAML blocks describing table relationships and join conditions	table_name, yaml_content, version, last_updated
business_glossary	Field-level SAP-to-English mapping with business context	sap_field, business_name, description, domain
action_tool_registry	Registered API tools the agent can invoke for SAP actions	tool_name, description, api_endpoint, http_method
agent_execution_limits	Policy thresholds that gate agent actions (amounts, regions)	policy_name, threshold, action_if_exceeded
action_audit_log	Immutable log of every agent action attempt (success/failure)	timestamp, agent_id, tool_name, status, payload_hash
action_api_spec	OpenAPI specifications cached from Azure APIM	api_name, openapi_json, version, sync_timestamp

Figure 9.1 — Agent Governance Data Model

10. AZURE AI FOUNDRY CROSS-PLATFORM INTEGRATION

Azure AI Foundry orchestrates the reasoning and action execution for agentic workflows. It connects to Databricks Unity Catalog for data and metadata, and routes validated actions through Azure APIM to SAP production systems.

10.1 Three Interaction Modes

Mode	When Used	How It Works
RAG (Retrieval)	Agent needs business context to answer a question	Queries Databricks Vector Search for SAP semantic dictionary, text tables, and glossary entries
Text-to-SQL (Structured)	User asks a data question (e.g., "open orders > \$50K")	Agent reads YAML schema manifest, generates SQL, executes via Serverless SQL Warehouse
Function Calling (Action)	User requests a change (e.g., "clear credit block")	Agent reads tool_registry, builds JSON payload, validates via guardrails, calls APIM → SAP

Figure 10.1 — AI Foundry Interaction Modes

10.2 Action Execution Lifecycle

- **Step 1 — Read Tool Definitions:** AI Foundry caches the OpenAPI specification matrix from `sap_enterprise.agent_governance.action_api_spec` to understand available system tools.
- **Step 2 — Generate Action Intent:** When the agent decides to take action, it generates a structured JSON block containing the required extraction variables (`customer_id`, `reason_code`, `requestor_id`).
- **Step 3 — Cross-Check Guardrails:** The JSON request payload is routed through a Python validation wrapper that queries `agent_execution_limits` to verify compliance and security thresholds.
- **Step 4 — Execute via APIM:** Once validated, AI Foundry calls an Azure APIM endpoint. APIM translates the JSON into an authenticated SAP OData web service or BAPI RFC, feeding the write request safely into SAP.
- **Step 5 — Audit Trail:** Every action attempt is logged to `action_audit_log` with timestamp, agent ID, tool name, status, and a hash of the payload for tamper detection.

11. IMPLEMENTATION ROADMAP & CHECKLIST

11.1 Phased Roadmap

Phase	Timeline	Key Deliverables
Phase 1: Foundation	Weeks 1-4	Create agent_governance catalog and schemas Deploy schema_manifests and business_glossary tables Identify top 10 CDS Views per module for extraction Document YAML schema guides for core domain tables
Phase 2: Tool Registry	Weeks 5-8	Build action_tool_registry with initial 5 tools Configure Azure APIM gateway linked to SAP OData Implement agent_execution_limits policy matrix Write Python guardrail validation functions
Phase 3: AI Integration	Weeks 9-12	Connect Azure AI Foundry to Unity Catalog metadata Deploy RAG pipeline with Vector Search indexes Enable Text-to-SQL with schema manifest consumption First agent pilot: sales order inquiry + credit block
Phase 4: Scale & Govern	Weeks 13-16	Expand tool registry to 20+ SAP actions Deploy action_audit_log with compliance dashboards Enable BDC Delta Sharing for automatic metadata sync Production rollout with full guardrail enforcement

Figure 11.1 — 16-Week Implementation Roadmap

11.2 Pre-Flight Checklist

Task	Owner	Status
Managed Identity federation configured (AI Foundry ↔ Databricks ↔ ADLS)	Platform Team	Required
CDS View inventory completed for target SAP modules (FI, SD, MM, PP)	SAP Basis / Data Eng	Required
English translation tables assembled (SAP text files merged with master data)	Domain Data Stewards	Required
Azure APIM gateway deployed and linked to SAP application router	Networking / Security	Required
YAML schema manifests authored for core domain tables (sales, finance)	Data Engineering	Required
Tool descriptions reviewed by business SMEs for accuracy and safety	Business + Security	Required
Guardrail policies validated against corporate compliance requirements	Legal / Compliance	Required
Audit logging pipeline tested end-to-end with tamper detection (payload hashing)	Security Engineering	Required

Figure 11.2 — Pre-Flight Implementation Checklist